

SESSION -5

Squares of a Sorted Array

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int i = 0;
        int j = nums.length - 1;
        int k = j;

        int[] newL = new int[nums.length];

        while (i < j) {
            int iSquare = nums[i] * nums[i];
            int jSquare = nums[j] * nums[j];

            if (iSquare > jSquare) {
                newL[k--] = iSquare;
                i++;
            } else {
                newL[k--] = jSquare;
                j--;
            }
        }

        newL[0] = nums[i] * nums[i];
        return newL;
    }
}
```

The screenshot displays a coding platform interface for the problem "977. Squares of a Sorted Array". The problem description states: "Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order." It includes two examples: Example 1 with input `[-4,-1,0,3,10]` and output `[0,1,9,16,100]`, and Example 2 with input `[-7,-3,2,3,11]` and output `[4,9,9,49,121]`. The solution is implemented in Java, and the test results show it is "Accepted" with a runtime of 0 ms. The code is as follows:

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3         int i = 0;
4         int j = nums.length - 1;
5         int k = j;
6
7         int[] newL = new int[nums.length];
8
9         while (i < j) {
10             int iSquare = nums[i] * nums[i];
11             int jSquare = nums[j] * nums[j];
12
13             if (iSquare > jSquare) {
14                 newL[k--] = iSquare;
15                 i++;
16             } else {
17                 newL[k--] = jSquare;
18                 j--;
19             }
20         }
21
22         newL[0] = nums[i] * nums[i];
23         return newL;
24     }
25 }
```